

Assignment One Report

Student Name	Omar Moustafa
Student ID	900222400
Student Email	omoustafa@aucegypt.edu

This is the report template for Assignment 1: **Multinomial**

Classification<https://github.com/nabadawy/DeepLearning/tree/master/Labs/lab2>

General Instructions:

- Your work on the assignment and answers in the report are **individual efforts**. Copying or any detected collaboration with a colleague will be severely penalized.
- In case you search for the answer to a question online (**only when explicitly mentioned to be allowed**), you must cite the reference or article used.
- Do not modify the report template and only fill in the missing parts as indicated by the "TODO" sections.
- Since the effort you need in organizing the report is minimal, **no grade is given for the report organization**.
- The report template is self-explanatory and has the same flow as the requirements in the provided Google Collaboratory notebook, so make sure to follow it while working on your experiments not to get stuck filling the report at the end. *The notebook will include mentions to what should be added to the report when necessary for you to double-check.*
- All the experiments and questions included in the report are **mandatory** and each have their own grade (**make sure you fill in all the required TODOs**)
- In case you want to add in extra experiments or work done other than what is required, feel free to do so but make sure to indicate that this is extra work. For instance, in case you are provided with a table to put in the results of experimenting with different parameters and you want to include another parameter other than those required, you can do so by adding extra rows to the table and highlighting them with a different color.
- Feel free to add any extra comments/conclusions/observations to any of the TODOs marked if you wanted.
- Your answers here regarding the experiments you did should be supported by graphs and plots generated using the Google Collaboratory notebooks, **otherwise will be considered plagiarism**.
- To submit the report, save it as a **PDF** file and upload it to the form provided in the instructions on the Blackboard. **Make sure to submit the code notebook as well, not just the report. (Failing to submit the notebook deduces 10 pts from the assignment grade).**

Part 1: Dataset Loading and Preparation (5 pts)

- After looking into the code in the dataset loading part at the beginning of the notebook, answer the following questions.
- Questions on Understanding (5 pts)
 1. **1.1) Question:** Why did we divide the images by 255 after loading the MNIST dataset? (5 pts)

My Answer:

The images were divided by 255 after loading the MNIST dataset to normalize the pixel values from their original range of $[0, 255]$ to the new and much smaller range that is $[0, 1]$. Therefore, originally, 0 equaled “black” and 255 equaled “white”, whereas now 0 = “black” and 1 equals “white.”

A couple of reasons why this first step is important is because it helps the neural networks train more efficiently and reliably, and also because many activation functions perform better when dealing with normalized inputs.

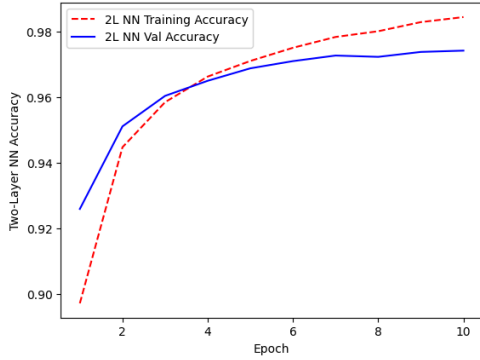
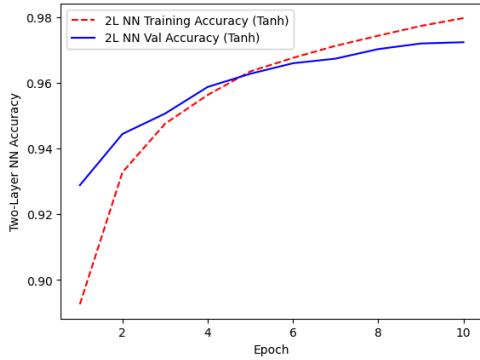
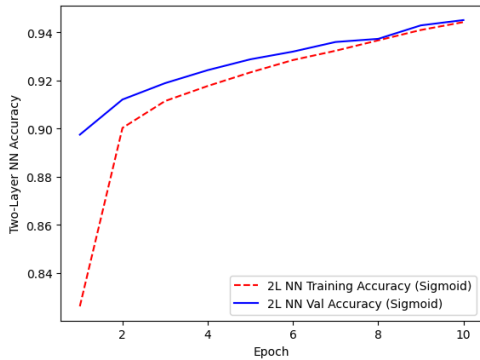
Part 2: Fully Connected Network (20 pts)

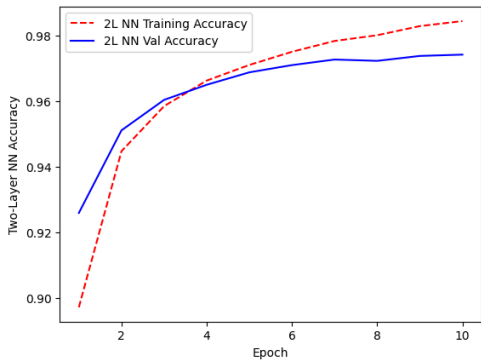
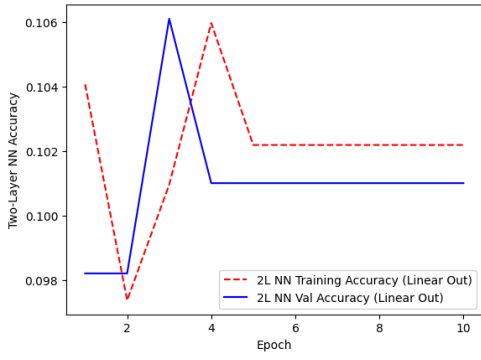
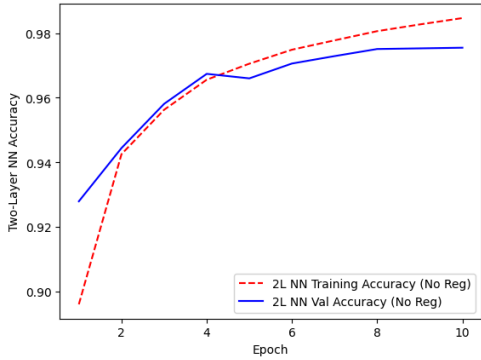
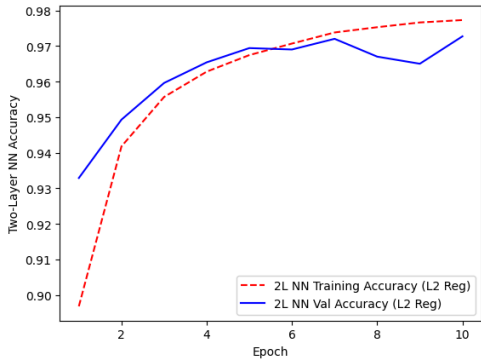
1. Experimental Results (20 pts)

Instructions on the Experiments:

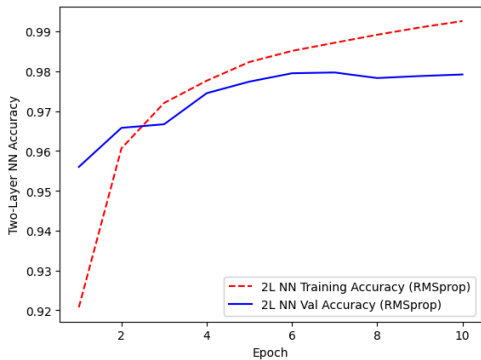
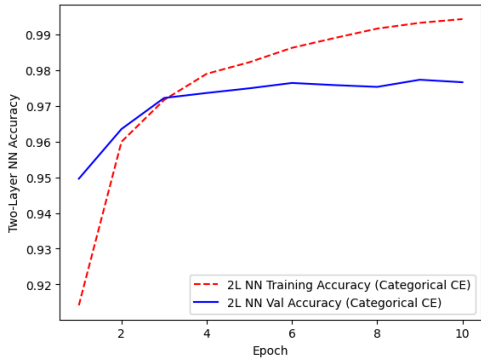
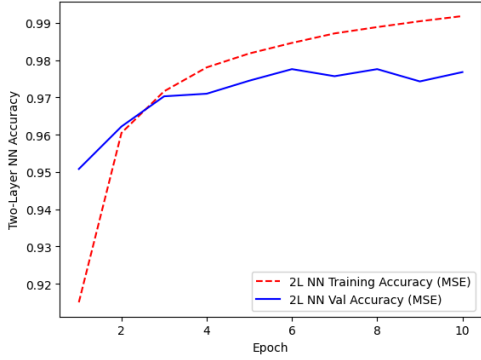
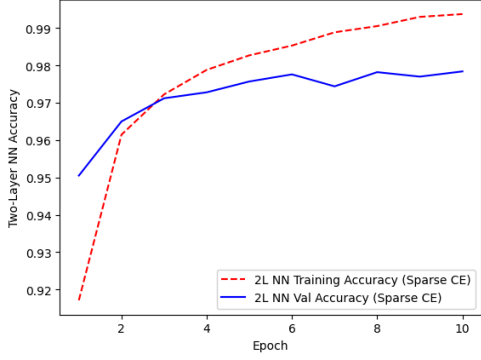
- The following table is composed of 5 columns, each with the following definition:
 - **Parameter:** The parameter/stage currently being tuned/changed in the experiments. For example, if it is required to change the activation function, the parameter would be “activation function”.
 - **Value:** The current value you tried for this parameter. For example for the optimizer and its hyperparameters the value can be SGD (learning rate = 0.1). **You are required to try out different values for the parameter in addition to its default value included in the notebook.**
 - **Plot:** the plot you obtained for the model evaluation when you used this specific parameter value.
 - **Accuracy:** The accuracy value indicated by the code cell printing it during training and evaluation of the model
 - **Overfitting or Underfitting:** Indicate whether the plot shows any overfitting/underfitting when using this parameter value. In case the plot doesn't indicate any overfitting or underfitting, type None.
- **Follow the following steps when doing your experiments:**
 - Complete the code in the notebook and run it using the default parameter values to get the results for these default values (plot and accuracy). **Include these results in the row corresponding to the first default parameter value and indicate any overfitting or underfitting (fill in the TODO cells in this row).**
 - Experiment with **each parameter at a time**. There is no need to experiment with different combinations of parameters/performing grid search not to waste time (experiment with each parameter on its own). Although performing grid search is better practice, you are only required to get your hands dirty in the assignment.
 - After experimenting with a parameter, you can **keep the parameter value giving the best results** and go ahead with experimenting with the **next parameter**. You can include the results obtained with the best value in the previous parameter as the results with the default value in the current parameter you are experimenting with. For example, after experimenting with the activation of the first layer, if you find that ReLU performed best, you can keep this value for the activation function of the first layer and include its corresponding results in the default value for the following experiments regarding the activation function of the output layer when you start experimenting with it.
- Each parameter has a number of required experiments indicated by the index after the “-” in the parameter name. Each experiment corresponds to trying out a different parameter value than those in the experiments before; with experiment “-1” corresponding to the default parameter value.
- For each optimizer, you need to experiment with different hyperparameters (at least two different experiments), so you will find for each optimizer experiment index, two sub-indices. For example, experiment 1.1 in optimizers is SGD with learning rate = 0.1, experiment 1.2 should be SGD with other learning rate and hyperparameter values and so on.

- • **Make sure to perform at least the indicated experiments by filling in all required TODOs in the table to avoid losing grades.**
- • There are **5 pts** on completing the fully connected network code and obtaining the first results with the default values.

Parameter	Value	Plot	Accuracy	Overfitting or Underfitting?
2.1) Activation Function First Layer - 1	<u>Default:</u> <u>ReLU</u>		0.9742	Overfitting
2.2) Activation Function First Layer – 2 (1 pts)	Tanh		0.9723	Overfitting
2.3) Activation Function First Layer – 3 (1 pts)	Sigmoid		0.9451	Underfitting

2.4) Activation Function Output Layer - 1	<u>Default:</u> <u>Softmax</u>		0.9742	Overfitting
2.5) Activation Function Output Layer – 2 (1 pts)	Linear		0.1010	Underfitting
2.6) Regularizer – 1	<u>Default:</u> <u>None</u>		0.9755	Overfitting
2.7) Regularizer – 2 (1 pts)	L2 Regularizer		0.9727	Overfitting

2.8) Regularizer – 3 (1 pts)	Dropout	Two-Layer NN Accuracy vs Epoch (Dropout)	0.9750	Underfitting
2.9) Optimizer (hyperparameter) – 1.1	<u>Default:</u> <u>SGD</u> <u>(learning rate = 0.1)</u>	Two-Layer NN Accuracy vs Epoch (SGD lr=0.1)	0.9756	Overfitting
2.10) Optimizer (hyperparameter) –1.2 (2 pts)	SGD (learning rate = 0.01)	Two-Layer NN Accuracy vs Epoch (SGD lr=0.01)	0.9384	Underfitting
2.11) Optimizer (hyperparameter) –2.1 (2 pts)	ADAM (learning rate = 0.001)	Two-Layer NN Accuracy vs Epoch (Adam)	0.9754	Overfitting

2.12) Optimizer (hyperparameter) –2.2 (2 pts)	RMSprop (learning rate = 0.001)		0.9792	Overfitting
2.13) Loss - 1	<u>Default:</u> <u>Crossentropy</u> <u>py</u>		0.9766	Overfitting
2.14) Loss – 2 (2 pts)	Mean Squared Error (MSE)		0.9768	Overfitting
2.15) Loss – 3 (2 pts)	Sparse Categorical Crossentropy		0.9784	Overfitting

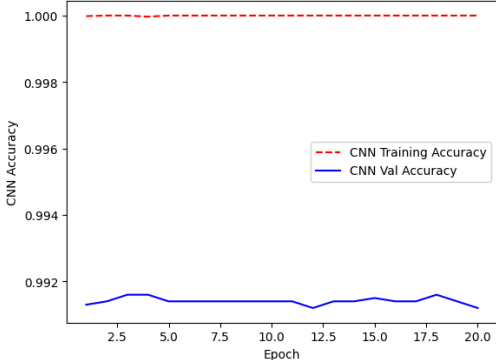
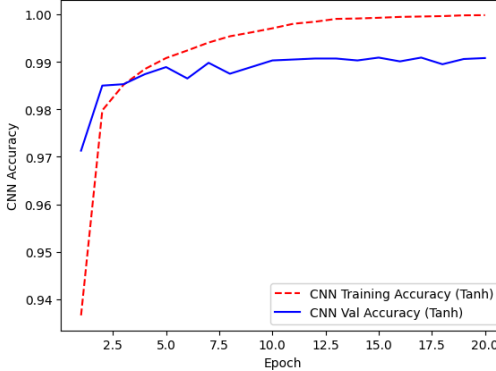
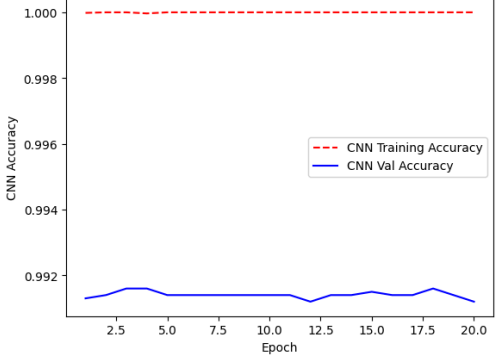
Part 3: Convolutional Neural Network (45 pts)

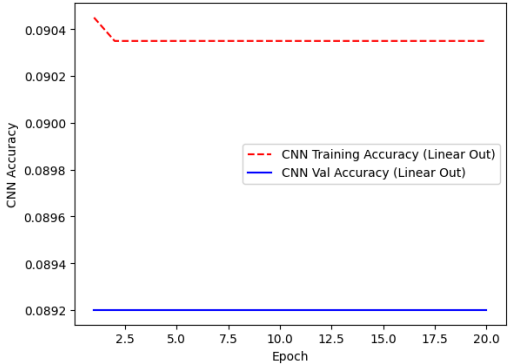
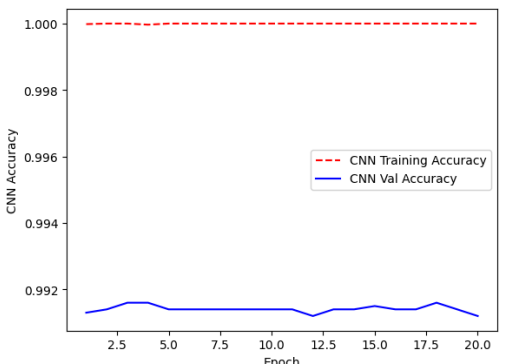
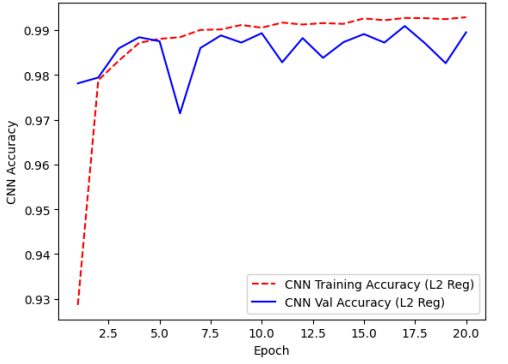
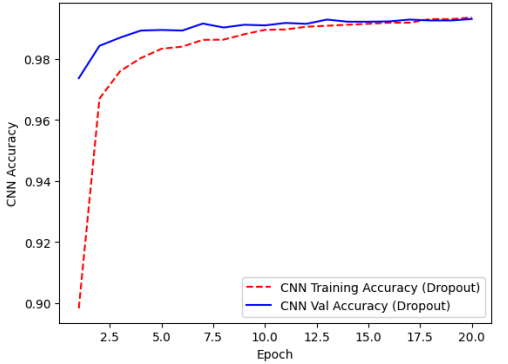
- Experimental Results (40 pts)

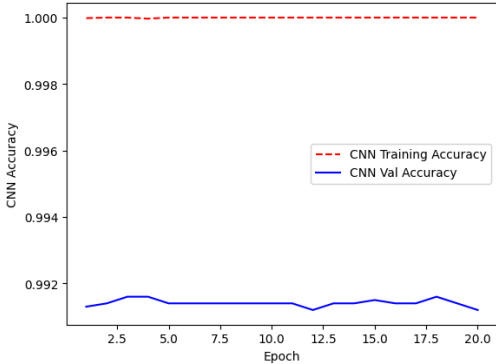
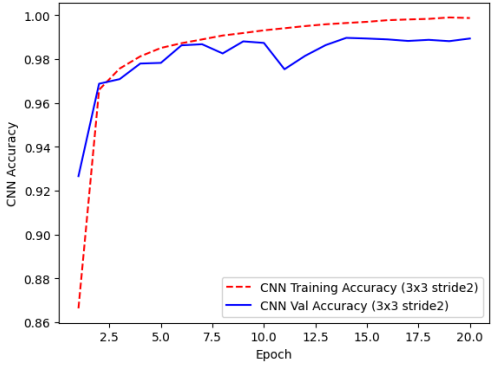
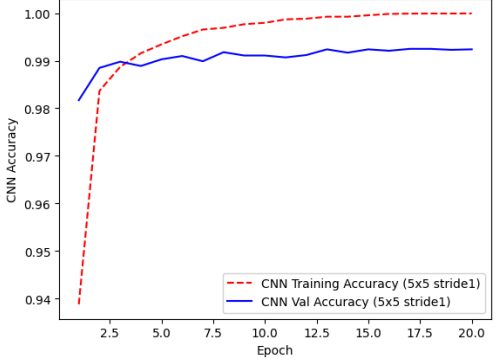
Instructions on the Experiments:

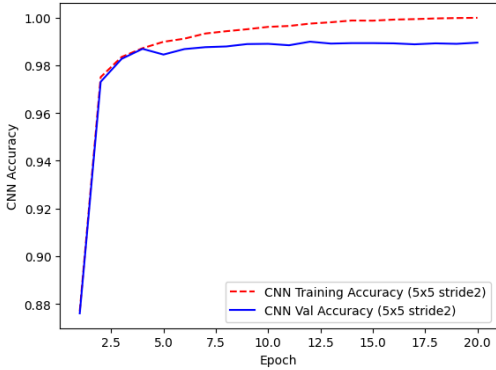
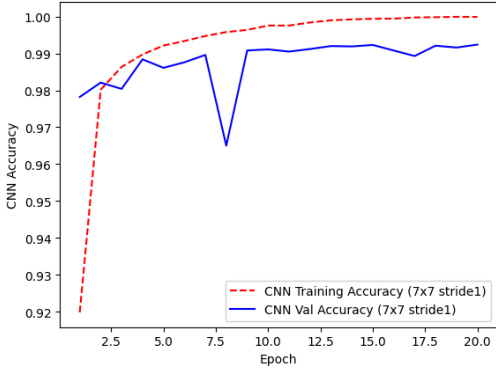
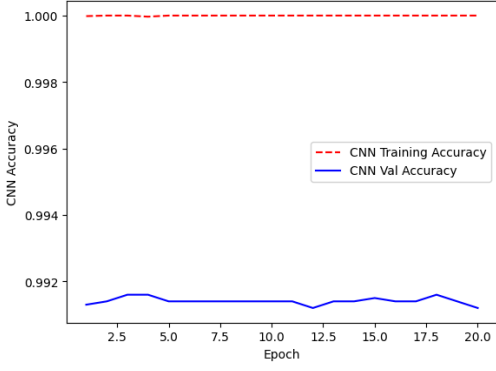
- The following table is composed of 5 columns, each with the following definition:
 - **Parameter:** The parameter/stage currently being tuned/changed in the experiments. For example, if it is required to change the activation function, the parameter would be “activation function”.
 - **Value:** The current value you tried for this parameter. For example for the optimizer and its hyperparameters the value can be SGD (learning rate = 0.1). **You are required to try out different values for the parameter in addition to its default value included in the notebook.**
 - **Plot:** the plot you obtained for the model evaluation when you used this specific parameter value.
 - **Accuracy:** The accuracy value indicated by the code cell printing it during training and evaluation of the model
 - **Overfitting or Underfitting:** Indicate whether the plot shows any overfitting/underfitting when using this parameter value. In case the plot doesn't indicate any overfitting or underfitting, type None.
- **Follow the following steps when doing your experiments:**
 - Complete the code in the notebook and run it using the default parameter values to get the results for these default values (plot and accuracy). **Include these results in the row corresponding to the first default parameter value and indicate any overfitting or underfitting (fill in the TODO cells in this row).**
 - Experiment with each parameter at a time. There is no need to experiment with different combinations of parameters/performing grid search not to waste time (experiment with each parameter on its own). Although performing grid search is better practice, you are only required to get your hands dirty in the assignment.
 - After experimenting with a parameter, you can **keep the parameter value giving the best results** and go ahead with experimenting with the **next parameter**. You can include the results obtained with the best value in the previous parameter as the results with the default value in the current parameter you are experimenting with. For example, after experimenting with the activation of the hidden layers, if you find that ReLU performed best, you can keep this value for the activation function of the hidden layers and include its corresponding results in the default value for the following experiments regarding the activation function of the output layer when you start experimenting with it.
- Each parameter has a number of required experiments indicated by the index after the “-” in the parameter name. Each experiment corresponds to trying out a different parameter value than those in the experiments before; with experiment “-1” corresponding to the default parameter value.
- For each optimizer, you need to experiment with different hyperparameters (at least two different experiments), so you will find for each optimizer experiment index, two sub-indices. For example, experiment 1.1 in optimizers is SGD with learning rate = 0.1, experiment 1.2 should be SGD with other learning rate and hyperparameter values and so on.

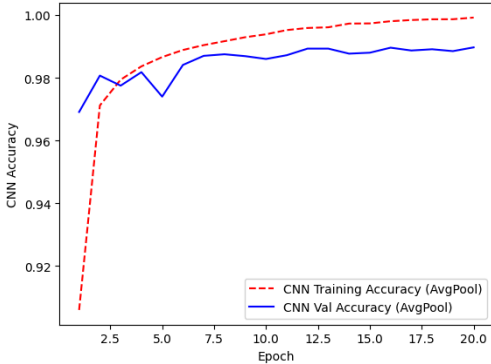
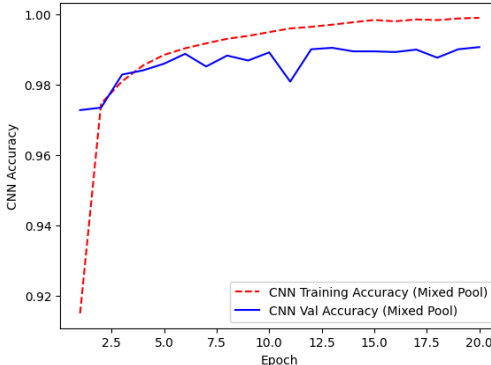
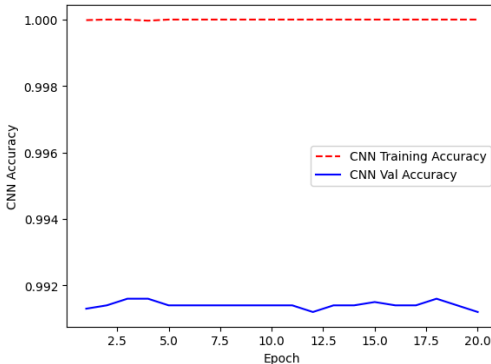
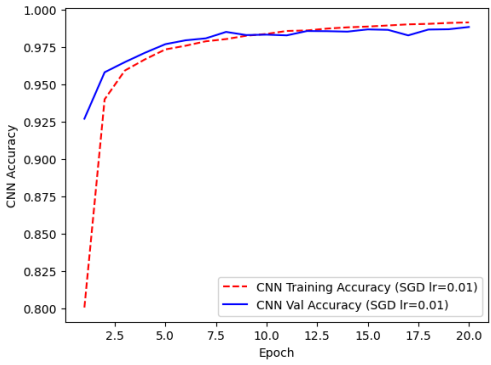
- Similar to the previous point, for each filter size, you need to experiment with different stride lengths
- ***Make sure to perform at least the indicated experiments by filling in all required TODOs in the table to avoid losing grades.***
- There are 5 pts on completing the convolutional neural network code and obtaining the first results with the default values.

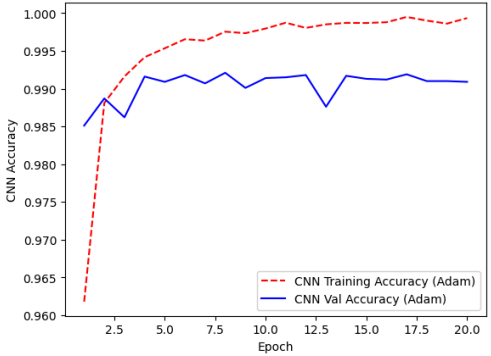
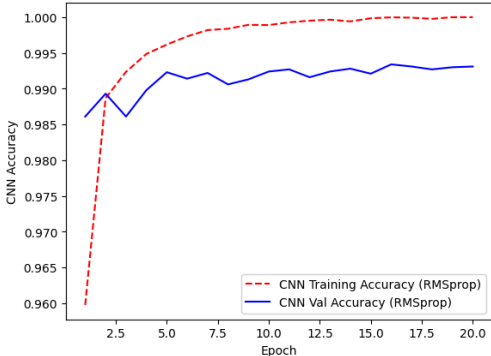
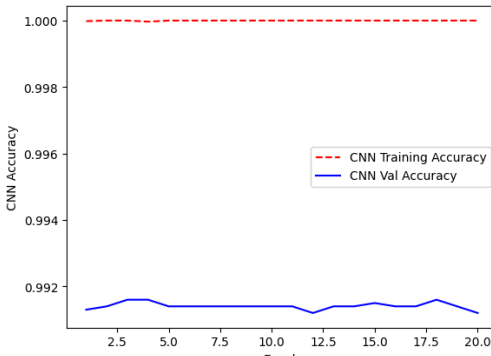
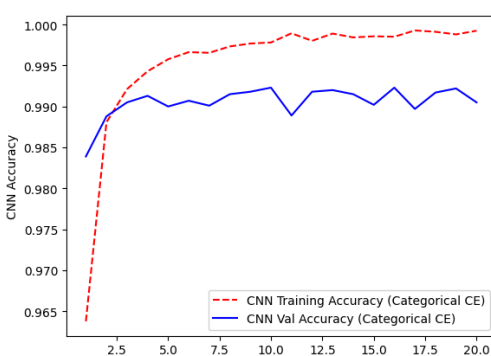
Parameter	Value	Plot	Accuracy	Overfitting or Underfitting
3.1) Activation Function Hidden Layers - 1	<u>Default:</u> <u>ReLU</u>		0.9912	Overfitting
3.2) Activation Function Hidden Layers – 2 (2.5 pts)	Tanh		0.9908	Overfitting
3.3) Activation Function Output Layer - 1	<u>Default:</u> <u>Softmax</u>		0.9912	Overfitting

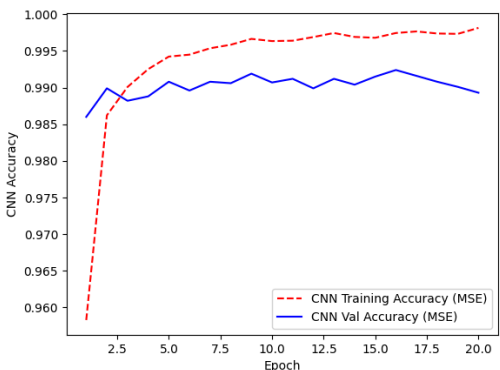
3.4) Activation Function Output Layer – 2 (2.5 pts)	Linear	 0.0892 Underfitting
3.5) Regularizer – 1	<u>Default:</u> <u>None</u>	 0.9912 Overfitting
3.6) Regularizer – 2 (2.5 pts)	L2 Regularization	 0.9895 Overfitting
3.7) Regularizer – 3 (2.5 pts)	Dropout	 0.9931 Neither

3.8) Filter Size (Stride length) – 1	<u>Default:</u> <u>3x3(strid</u> <u>e = 1)</u>	 CNN Accuracy vs Epoch Legend: CNN Training Accuracy (red dashed), CNN Val Accuracy (blue solid)	0.9912	Overfitting
3.9) Filter Size (Stride length) – 2 (2.5 pts)	3x3 (stride = 2)	 CNN Accuracy vs Epoch Legend: CNN Training Accuracy (3x3 stride2) (red dashed), CNN Val Accuracy (3x3 stride2) (blue solid)	0.9893	Overfitting
3.10) Filter Size (Stride length) – 3 (2.5 pts)	5x5(strid e=1)	 CNN Accuracy vs Epoch Legend: CNN Training Accuracy (5x5 stride1) (red dashed), CNN Val Accuracy (5x5 stride1) (blue solid)	0.9924	Overfitting

3.11) Filter Size (Stride length) – 4 (2.5 pts)	5x5(strid e=2)	 CNN Accuracy vs Epoch for 5x5 filter, stride 2. Training accuracy (red dashed) reaches ~0.99. Validation accuracy (blue solid) reaches ~0.99.	0.9895	Overfitting
3.12) Filter Size (Stride length) – 5 (2.5 pts)	7x7(strid e=1)	 CNN Accuracy vs Epoch for 7x7 filter, stride 1. Training accuracy (red dashed) reaches ~0.99. Validation accuracy (blue solid) reaches ~0.99.	0.9924	Overfitting
3.13) The Two Pooling Layers – 1	<u>Default</u> ≡ <u>Maxpool</u> ↕ <u>Maxpool</u>	 CNN Accuracy vs Epoch for Default Maxpool. Training accuracy (red dashed) is 1.00. Validation accuracy (blue solid) is ~0.991.	0.9912	Overfitting

3.14) The Two Pooling Layers – 2 (2.5 pts)	AvgPool , AvgPool	 CNN Accuracy vs Epoch (AvgPool)	0.9897	Overfitting
3.15) The Two Pooling Layers – 3 (2.5 pts)	MaxPool , AvgPool	 CNN Accuracy vs Epoch (Mixed Pool)	0.9907	Overfitting
3.16) Optimizer (hyperparameter) – 1.1	<u>Default:</u> <u>SGD</u> (learning rate = 0.1)	 CNN Accuracy vs Epoch	0.9912	Overfitting
3.17) Optimizer (hyperparameter) – 1.2 (2 pts)	SGD (learning rate = 0.01)	 CNN Accuracy vs Epoch (SGD lr=0.01)	0.9883	Overfitting

3.18) Optimizer (hyperparameter) –2.1 (2 pts)	ADAM (learning rate = 0.01)		0.9909	Overfitting
3.19) Optimizer (hyperparameter) –2.2 (2 pts)	RMSprop (learning rate = 0.01)		0.9931	Overfitting
3.20) Loss - 1	<u>Default:</u> <u>Crossentropy</u>		0.9912	Overfitting
3.21) Loss – 2 (2 pts)	Categorical Crossentropy		0.9905	Overfitting

3.22) Loss – 3 (2 pts)	Mean Squared Error (MSE)		0.9893	Overfitting
---------------------------	--------------------------	--	--------	-------------

• Questions on Understanding (5 pts)

1. **3.23) Question:** How does the best accuracy achieved by CNN compare to the Fully Connected Network? Why do you think this is the case? (5 pts)

My Answer:

- Comparison of the Best Accuracies from Parts 2 & 3:
 - **Fully Connected Network Best Accuracy:** 0.9792 (97.92%) from 2.12 with RMSprop (learning rate = 0.001) as an optimizer.
 - **Convolutional Neural Network Best Accuracy:** 0.9931 (99.31%) from 3.7 with Dropout.
 - Therefore, the best achieved accuracy of the CNN was about 1.39% higher than that of the Fully Connected Network.
- This is the case because of CNNs more advanced traits including Translation Invariance, Parameter Efficiency, and Hierarchical Feature Learning, to name a few. These traits and reasons can be described as follows:
 - **Translation Invariance:** CNNs recognize patterns anywhere in the image through the use of shared filters, whereas Fully Connected Networks treat each pixel position independently.
 - **Parameter Efficiency:** CNNs use far less parameters through weight sharing and localization, which help reduce overfitting and also improve the model generalization. On the contrary, Fully Connected Networks tend to fail with images and end up leading to computational waste and overfitting.
 - **Preservation of Spatial Relationships:** CNNs maintain the

two-dimensional structure of images through assigned convolutional operations, where the early layers detect simple features (e.g., edges and textures) and the deeper layers detect complex features (e.g., entire shapes and objects). On the other hand, Fully Connected Networks work to flatten the input image leading to a major loss of important spatial information.

Part 4: Machine Learning Interpretability (15 pts)

There are **5 pts** on completing the SHAP code correctly and including its output in the submitted notebook.

1. Questions on Understanding (5 pts)

4.1) Question: What is the benefit of machine learning interpretability? (5 pts)

My Answer:

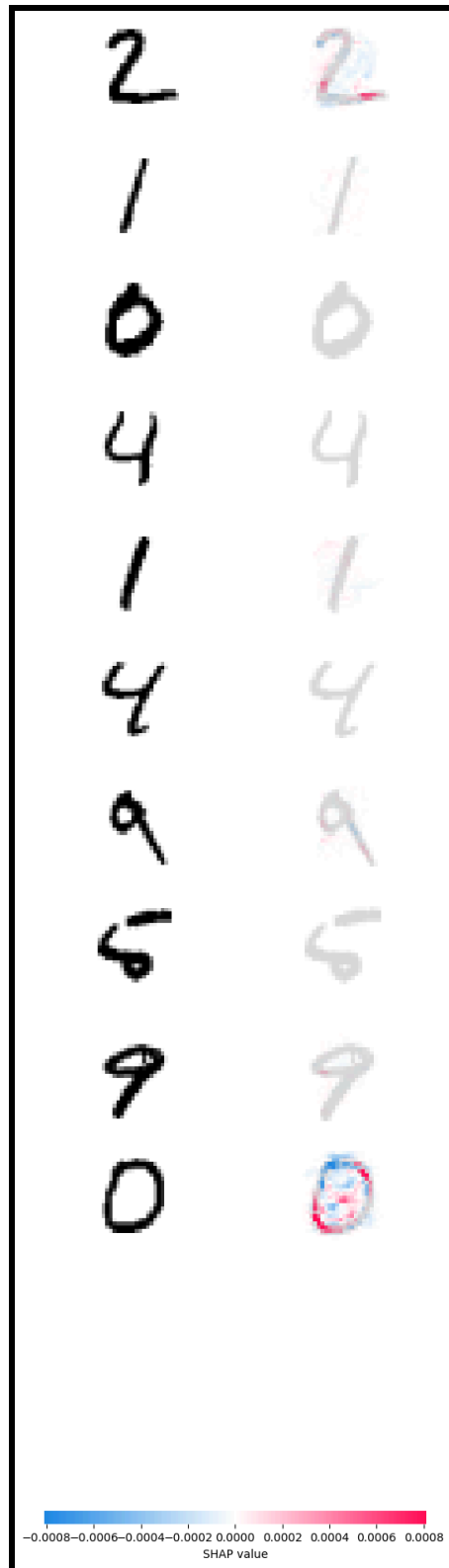
One important benefit of machine learning interpretability is selecting and engineering features. “Interpretable models can assist in selecting and engineering features by determining which elements are most important for generating predictions. The model’s performance can be increased or improved using this information.”

<https://www.pranathiss.com/blog/importance-of-machine-learning/>

Another major benefit of machine learning interpretability is ease in informed decision making. “The more interpretable a machine learning model, the easier it is for someone to understand why certain decisions or predictions were made. A model is more interpretable than another model if its decisions are easier for a human to understand than the other model’s decisions.”

<https://christophm.github.io/interpretable-ml-book/interpretability.html>

SHAP Plot and Comments on It (5 pts)



My Comments on the SHAP Plot:

The SHAP plot above represents the focus of the CNN on semantically significant features for digit classification. The red regions (positive contributions) consistently highlight the stroke patterns of each digit while the blue regions (negative contributions) highlight the lack of ink in areas that are meant to help distinguish between similar digit classifications.

Some key observations include the following:

- The model successfully identified the main structural features of each and every one of the numeric digits. All of the digits are distinguishable and easily identifiable as the number they are meant to be.
- There are only two instances where there was focus on the background pixels (for the digits 2 and 0). This very minimal focus on irrelevant regions indicates that the model is generalizing very well and focusing on the meaningful features.
- The model's feature importance seems to align with human perception by focusing on the distinctive visual aspects that are commonly used to differentiate digits from each other, such as stroke directions and curvature points, to name a couple.

Therefore, the CNN successfully learned to recognize the fundamental visual traits that define the identity of each numeric digit, demonstrating human-like recognition instead of depending on random pixel correlations.